

Generated benchmark artifacts

Typeset from `paper/generated/aarch64/` by `benchmarks/scripts/build_pdf.py`. Every number resolves to a row in the result set named below; see `PROVENANCE.md` for the inputs, transformation and caveats of each artifact.

```
result set: current
commit: 18d0f83627b5
      host: galaxy
measured: 2026-09-12
inputs: sha256:f9529abbd784055c
```

Dataset	Tool	Metric	-@	Mapped	Mapq 60	Missed (%)	Wrong Q60	Index (s)	Map (s)	Mem (GB)
B01	shmap-rs	Containment	1	149 194	139 309	6.78	—	6.53	37.47	2.66
B01	shmap-rs	Jaccard	1	146 120	138 421	7.37	—	6.50	52.48	2.65
B01	shmap-rs	bucket_SH	1	149 236	138 250	7.49	—	6.50	29.74	2.65
B01	cpp-shmap	Containment	1	149 194	139 309	6.78	—	25.42	77.67	18.85
B01	cpp-shmap	Jaccard	1	146 120	138 421	7.37	—	25.34	105.03	18.85
B01	cpp-shmap	bucket_SH	1	149 236	138 250	7.49	—	25.64	61.44	18.85
B01	winnowmap2	—	32	189 022	—	—	—	—	1493.40	9.83
B02	shmap-rs	Containment	1	125 000	117 532	5.97	0	6.54	29.23	2.66
B02	shmap-rs	Jaccard	1	125 000	119 065	4.75	6	6.51	45.48	2.65
B02	shmap-rs	bucket_SH	1	125 000	116 800	6.56	1	6.51	25.20	2.65
B02	cpp-shmap	Containment	1	125 000	117 532	5.97	—	25.55	61.58	18.85
B02	cpp-shmap	Jaccard	1	125 000	119 065	4.75	—	25.47	90.24	18.85
B02	cpp-shmap	bucket_SH	1	125 000	116 802	6.56	—	25.43	54.99	18.85
B02	winnowmap2	—	32	129 488	—	—	—	—	1123.80	9.47
B03	shmap-rs	Containment	1	241 988	227 521	6.19	—	6.52	49.90	2.65
B03	shmap-rs	Jaccard	1	241 262	228 887	5.63	—	6.50	66.00	2.65
B03	shmap-rs	bucket_SH	1	242 047	226 931	6.43	—	6.48	41.22	2.65
B03	cpp-shmap	Containment	1	241 991	227 521	6.19	—	25.57	86.53	18.85
B03	cpp-shmap	Jaccard	1	241 265	228 886	5.63	—	25.68	117.10	18.85
B03	cpp-shmap	bucket_SH	1	242 050	226 934	6.43	—	26.06	69.73	18.85
B03	winnowmap2	—	32	280 093	—	—	—	—	1113.10	8.22
B04	shmap-rs	Containment	1	2 419 767	2 273 103	6.28	—	6.49	502.96	2.65
B04	shmap-rs	Jaccard	1	2 411 433	2 286 067	5.74	—	6.54	673.99	2.66
B04	shmap-rs	bucket_SH	1	2 420 302	2 267 208	6.52	—	6.51	408.03	2.65
B04	cpp-shmap	Containment	1	2 419 796	2 273 121	6.28	—	25.04	877.05	18.85
B04	cpp-shmap	Jaccard	1	2 411 461	2 286 083	5.74	—	25.12	1195.37	18.85
B04	cpp-shmap	bucket_SH	1	2 420 331	2 267 238	6.52	—	25.12	703.13	18.85
B04	winnowmap2	—	32	2 808 979	—	—	—	—	10434.70	8.47
B05	shmap-rs	Containment	1	39 608	37 839	58.97	—	6.52	13.64	2.55
B05	shmap-rs	Jaccard	1	5 956	5 409	94.13	—	6.53	17.35	2.66
B05	shmap-rs	bucket_SH	1	41 115	39 226	57.46	—	6.50	12.84	2.65
B05	cpp-shmap	Containment	1	39 608	37 839	58.97	—	25.27	26.72	18.84
B05	cpp-shmap	Jaccard	1	5 956	5 409	94.13	—	25.24	31.51	18.85
B05	cpp-shmap	bucket_SH	1	41 115	39 225	57.47	—	25.43	24.95	18.85
B05	winnowmap2	—	32	99 107	—	—	—	—	327.10	6.11
B06	bwa-mem2	—	32	41 807 925	—	—	—	—	565.20	35.66
B07	bwa-mem2	—	32	83 600 076	—	—	—	—	1061.90	35.66
B08	bwa-mem2	—	32	146 287 615 ²	—	—	—	—	1822.80	35.66
B09	bwa-mem2	—	32	20 638 443	—	—	—	—	264.50	24.06

Dataset	Metric	Matches per read			Potential		Buckets per read	
		possible	examined	in mapping	realized	unrealized	seeded	final
B01	Containment	113 462	2 376	206	47.8×	11.6×	194.0	2.38
B01	Jaccard	113 462	2 376	203	47.8×	11.7×	194.0	2.31
B01	bucket.SH	113 462	2 376	208	47.8×	11.4×	194.0	2.28
B02	Containment	113 804	2 907	209	39.2×	13.9×	206.2	2.29
B02	Jaccard	113 804	2 907	208	39.2×	13.9×	206.2	2.29
B02	bucket.SH	113 804	2 907	211	39.2×	13.8×	206.2	2.26
B03	Containment	70 179	1 886	117	37.2×	16.1×	251.7	2.35
B03	Jaccard	70 179	1 886	117	37.2×	16.1×	251.7	2.34
B03	bucket.SH	70 179	1 886	119	37.2×	15.9×	251.7	2.24
B04	Containment	69 463	1 938	116	35.8×	16.6×	250.1	2.35
B04	Jaccard	69 463	1 938	116	35.8×	16.7×	250.1	2.34
B04	bucket.SH	69 463	1 938	118	35.8×	16.4×	250.1	2.25
B05	Containment	67 358	307	51	219.6×	6.0×	84.8	0.99
B05	Jaccard	67 358	307	9	219.6×	33.1×	84.8	0.17
B05	bucket.SH	67 358	307	54	219.6×	5.7×	84.8	1.00

Table 2: Efficiency of the seed heuristic. Realized potential is possible matches divided by matches examined (work avoided); unrealized potential is matches examined divided by matches inside the reported mapping (work still wasted).

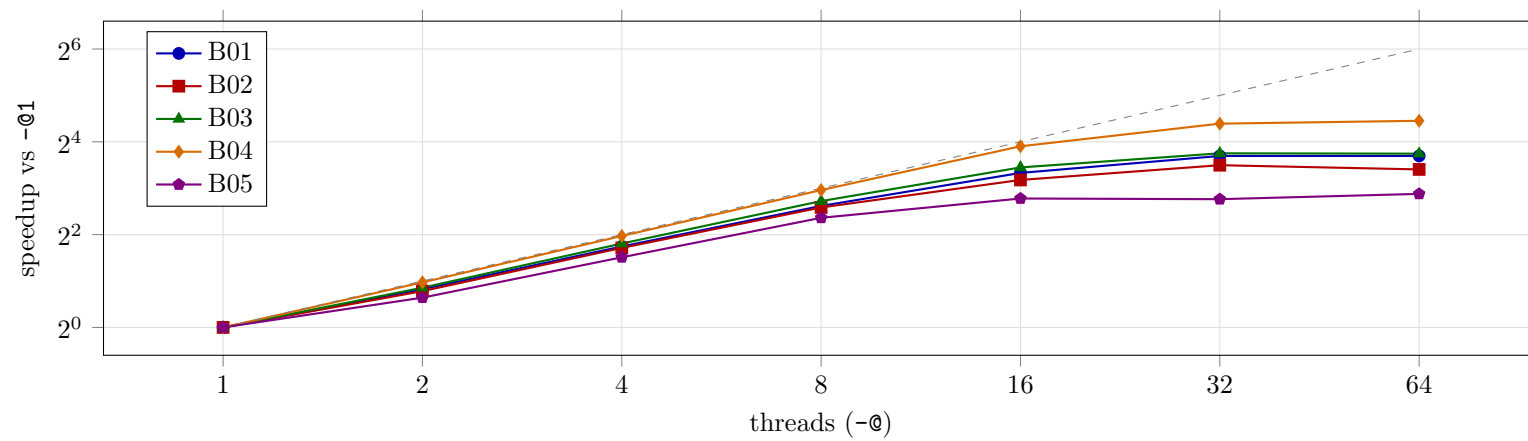


Figure 1: Whole-run speedup against thread count, Containment. The dashed line is linear speedup.

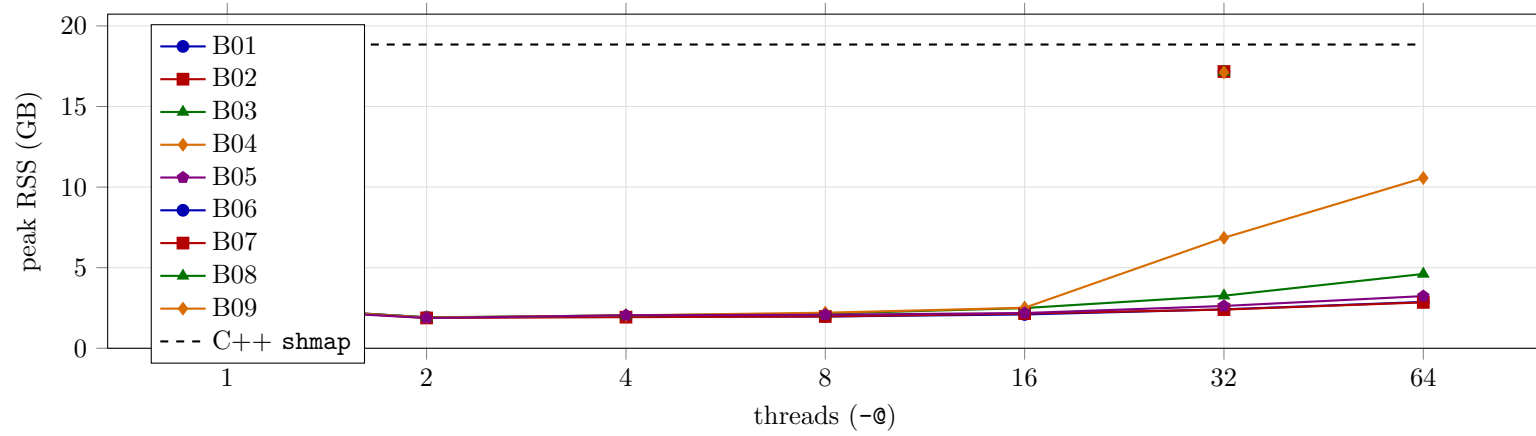


Figure 2: Peak resident memory against thread count, Containment, with the C++ reference as a horizontal line.

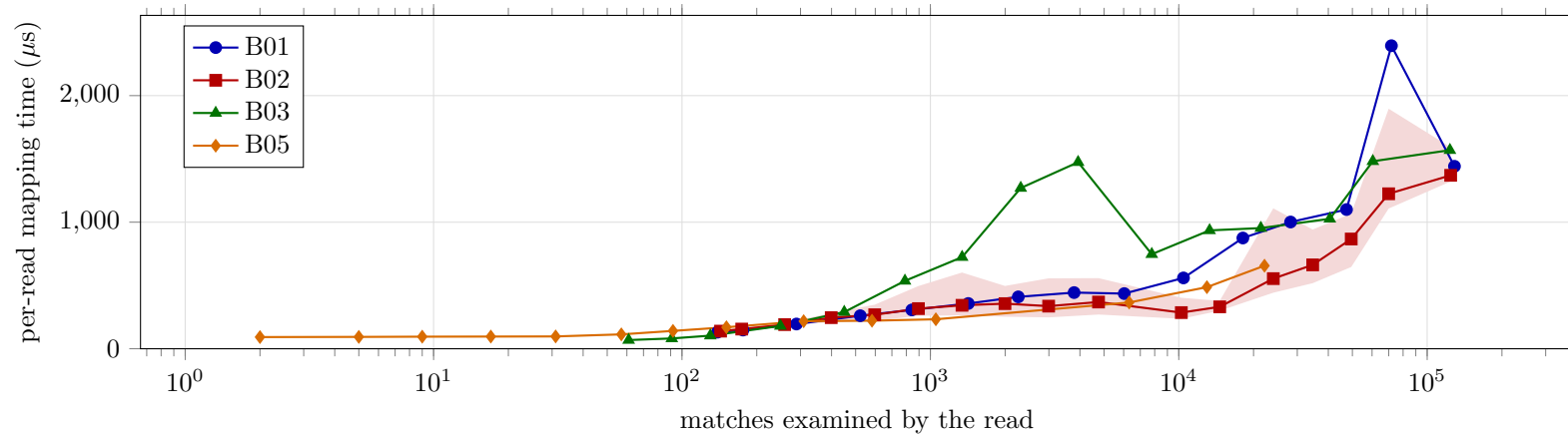


Figure 3: Per-read mapping time against the number of matches the read examined, Containment. Points are per-bin medians over reads grouped into 18 logarithmic bins; the shaded band is the interquartile range for the simulated set.

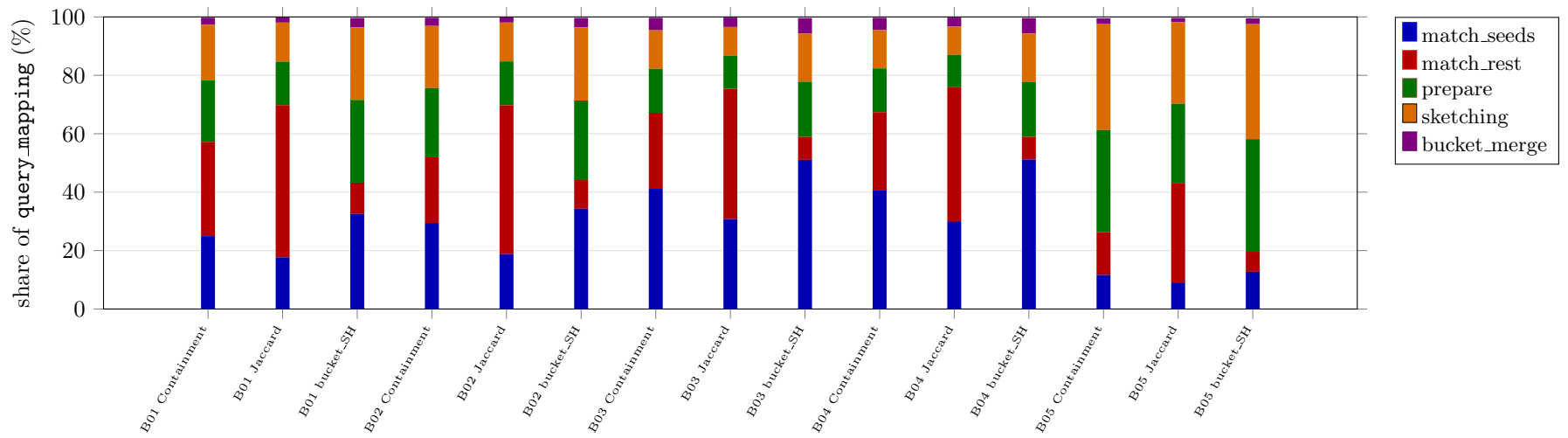


Figure 4: Where mapping time goes: stage shares of `query_mapping`, single-threaded.

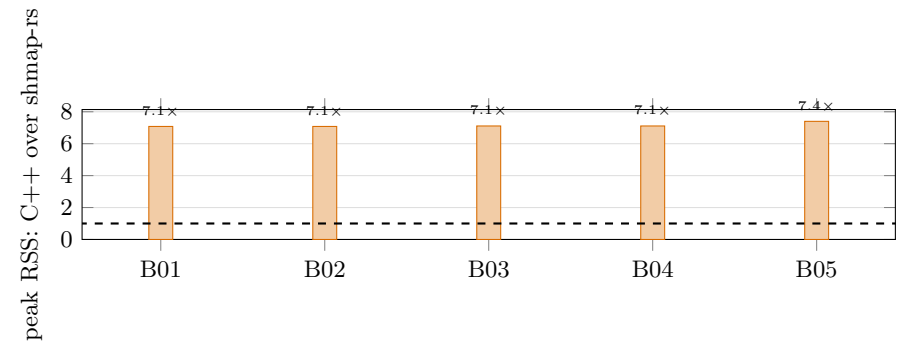
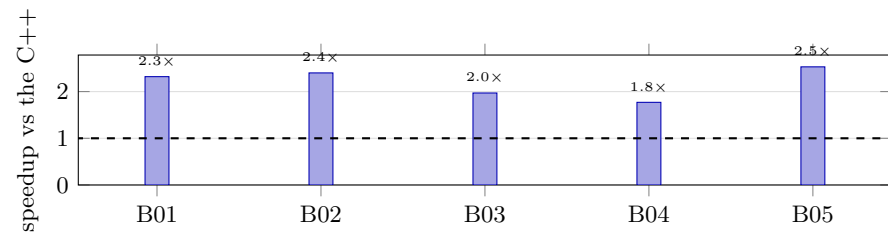


Figure 5: The two headline results, per benchmark, single-threaded on both sides: wall-clock speedup over the C++ (top) and how many times less peak memory the port uses (bottom). The dashed line is parity.

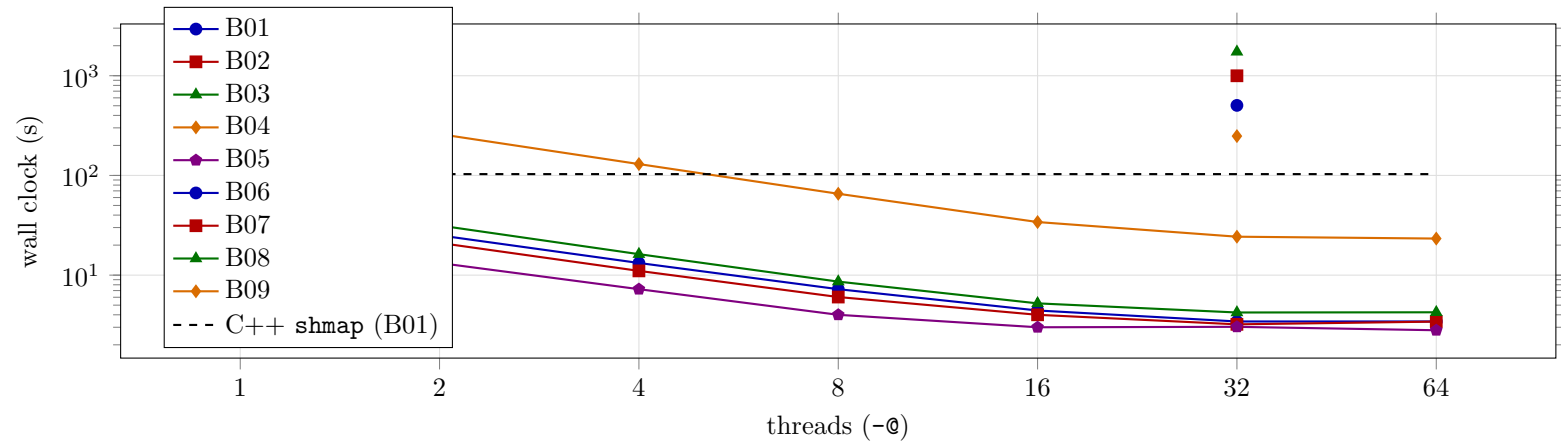


Figure 6: Wall clock against thread count, Containment, with the C++ reference as a horizontal line. The gap at 2^1 is the single-threaded speedup the summary quotes; everything to the right of it is threading the C++ does not have.